基于下一代硬件的Ascend C Reg矢量编程

作者 田宏民

时间 2026/03

目录

Part 1 课程背景和目标

Part 2 下一代Regbase抽象硬件架构

Part 3 下一代Reg矢量编程模型

Part 4 下一代Reg矢量编程API

Part 5 下一代Reg矢量编程优化技巧

Part 6 当代矢量算子与下一代对比

课程背景和目标

背景:

- 在下一代昇腾处理器中，向开发者全新开放 AIV 核的SIMD Reg寄存器。开发者可直接基于SIMD Reg寄存器作为进行数据交换，开发定制化 Kernel 代码，能更灵活地利用硬件底层资源。

课程目标:

- 掌握Regbase架构下的Reg矢量编程

CANN

目录

Part 1 课程背景和目标

Part 2 下一代Regbase抽象硬件架构

Part 3 下一代Reg矢量编程模型

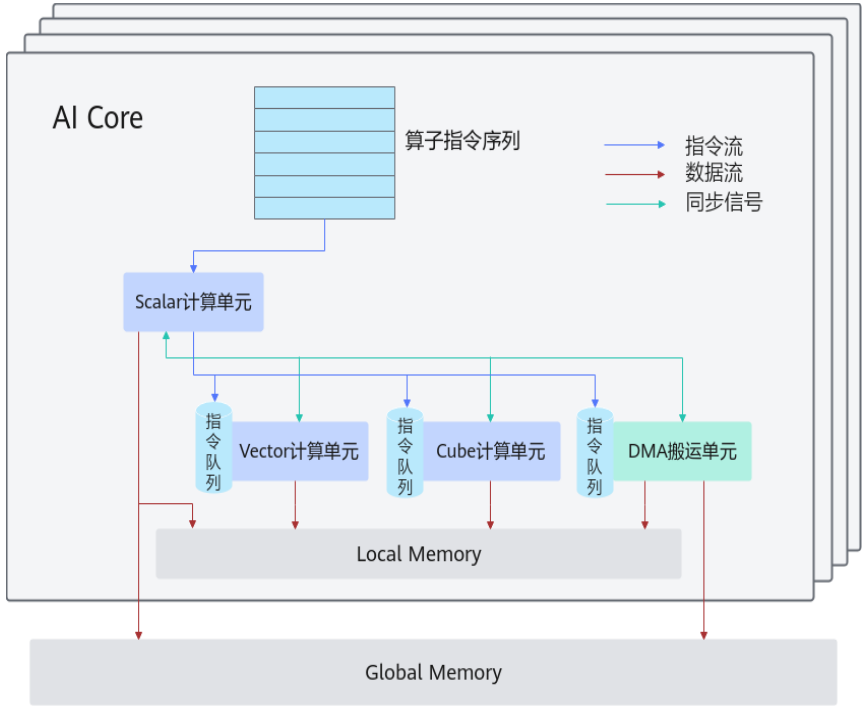
Part 4 下一代Reg矢量编程API

Part 5 下一代Reg矢量编程优化技巧

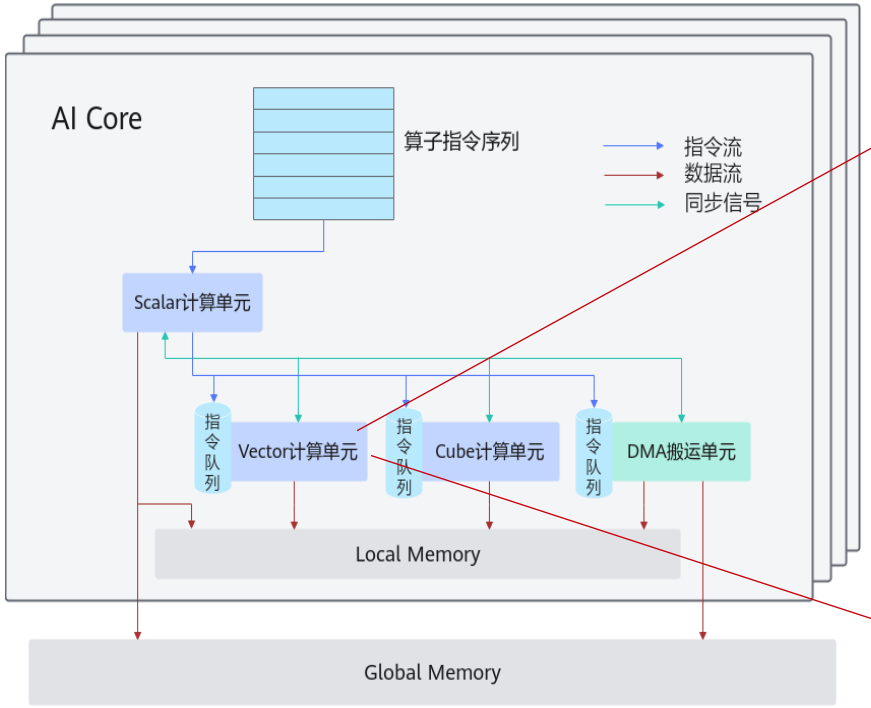
Part 6 当代矢量算子与下一代对比

下一代Regbase抽象硬件架构

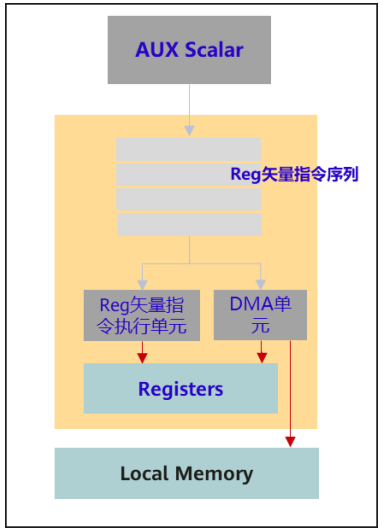
当代SIMD抽象硬件架构



下一代 Regbase抽象硬件架构



SIMD Vector计算单元



- **变动点：**
  - SIMD Vector新增Reg内存层级。
  - Vector计算单元单Block内调用SIMD VF（Vector Function）；
  - SIMD Vector内部新增AUX Scalar，该Scalar单元仅负责SIMD VF函数内部标量计算，能力相比“Scalar计算单元”弱；

CANN

目录

Part 1 课程背景和目标

Part 2 下一代Regbase硬件抽象架构

Part 3 下一代Reg矢量编程模型

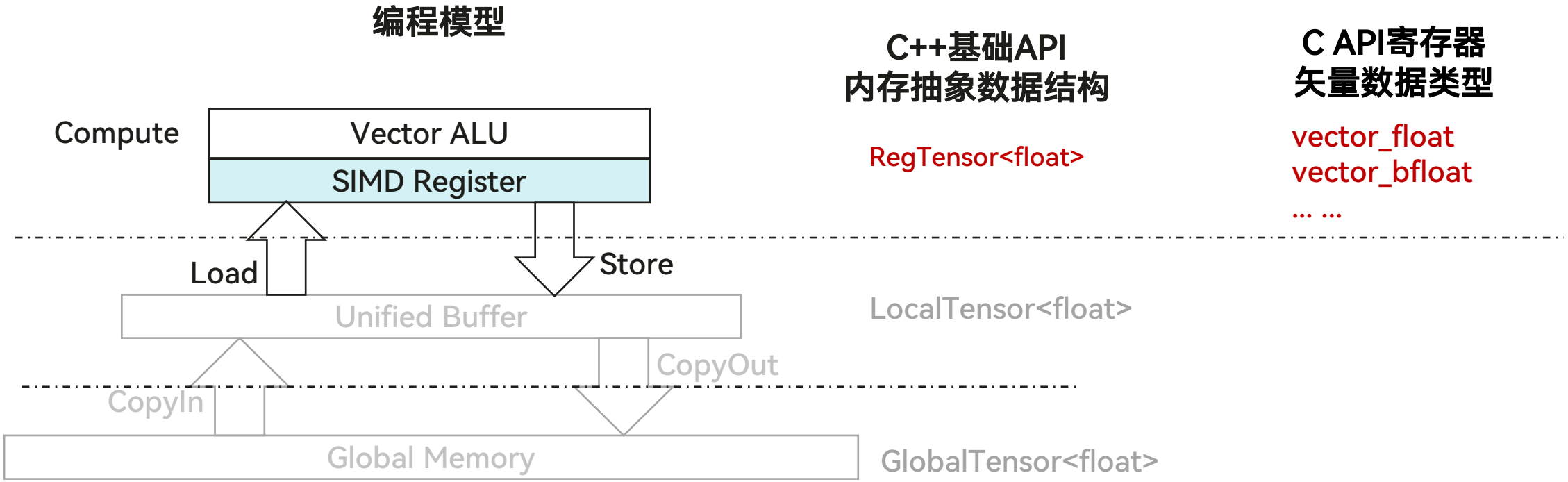
Part 4 下一代Reg矢量编程API

Part 5 下一代Reg矢量编程优化技巧

Part 6 当代矢量算子与下一代对比

Reg矢量编程模型 内存与数据流

- 单个SIMD Reg寄存器的长度定义为VL(Vector Length），其值为 256 Bytes。
- C++基础API Reg矢量计算使用RegTensor<T>定义SIMD Reg寄存器
- C语言 API Reg矢量计算使用vector_float等定义SIMD Reg寄存器
- Reg矢量计算须统一放到一个函数中，此函数定义为SIMD VF(Vector Function)



Reg矢量编程模型 API调用

```
__global__ __aicore__ void __kernel (__gm__ float * x, __gm__ float * y) {
{
  ......
  AscendC::DataCopy(srcLocalTenosr, xGlobalTensor, size);
  __ubuf__ float* src0_addr = (__ubuf__ float*)srcLocalTenosr.GetPhyAddr();
  __ubuf__ float* dst_addr = (__ubuf__ float*)dstLocalTenosr.GetPhyAddr();
  asc_vf_call<add_vf>(tmp_addr, src0_addr, src1_addr);
  ......
  asc_vf_call<add_vf>(dst_addr, src0_addr, tmp_addr);
  ......
  AscendC::DataCopy(yGlobalTensor, dstLocalTenosr, size);
  ......
}
```

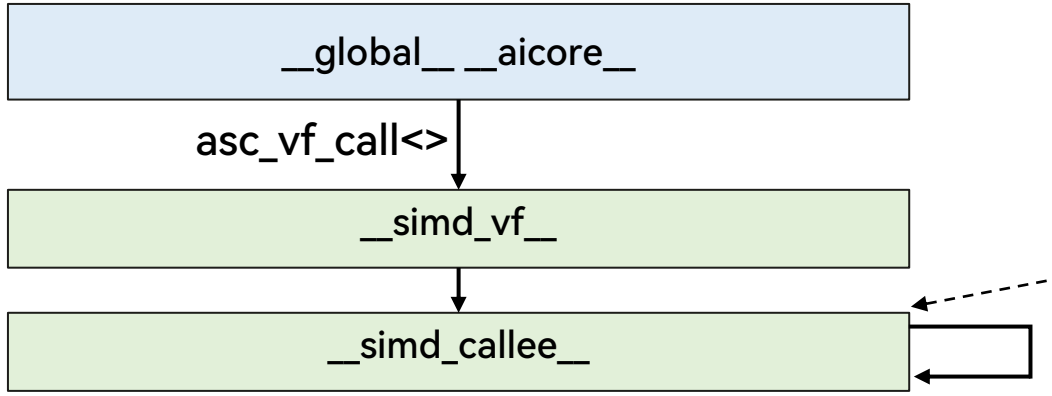
1 必须使用asc_vf_call来调用VF函数

2 VF(Vector Function)必须使用__simd_vf__修饰

```
__simd_vf__ inline void add_vf(__ubuf__ float* dst_addr,
  __ubuf__float* src0_addr, __ubuf__ float* src1_addr)
{
  ...
  add_vf_compute(dst_reg, src0_addr, src1_addr);
  asc_store(dst_addr , dst_reg, mask);
}
```

3 Reg矢量计算只能在VF内

```
__simd_callee__ inline void add_vf_compute(
   vector_f32 & dst_reg,
    __ubuf__ float* src0_addr,
    __ubuf__ float* src1_addr)
{
  ...
  asc_add(dst_reg, vreg0, vreg1, mask);
  ...
}
```



__simd_callee__修饰函数只能在VF内调用，其标量计算在AuxScalar执行

CANN

本章小结

- 下一代全新开放寄存器层级：
  - AIV的SIMD引入Reg架构，编程模型数据流GM<->UB<->Reg
  - C++基础API Reg矢量计算使用RegTenosr<>定义SIMD Reg寄存器
  - C语言 API Reg矢量计算使用vector_float、vector_bfloat16等定义SIMD Reg寄存器

- 编程模型进一步升级：
  - Reg矢量计算基本流程：
    1. Reg矢量计算需要在SIMD VF内部实现，使用__simd_vf__修饰
    2. 先在Kernel函数中将GM数据搬运到UB中，将UB地址作为参数传递给SIMD VF函数
    3. 然后在SIMD VF中，从UB读取数据到Reg，完成计算后将Reg数据写回到UB
    4. 最后在Kernel将计算结果从UB搬运到GM
    5. 其Kernel函数通过asc_vf_call调用SIMD VF

CANN

目录

Part 1 课程背景和目标

Part 2 下一代SIMD抽象Regbase硬件架构

Part 3下一代Reg矢量编程模型

Part 4 下一代Reg矢量编程API

Part 5 下一代Reg矢量编程优化技巧

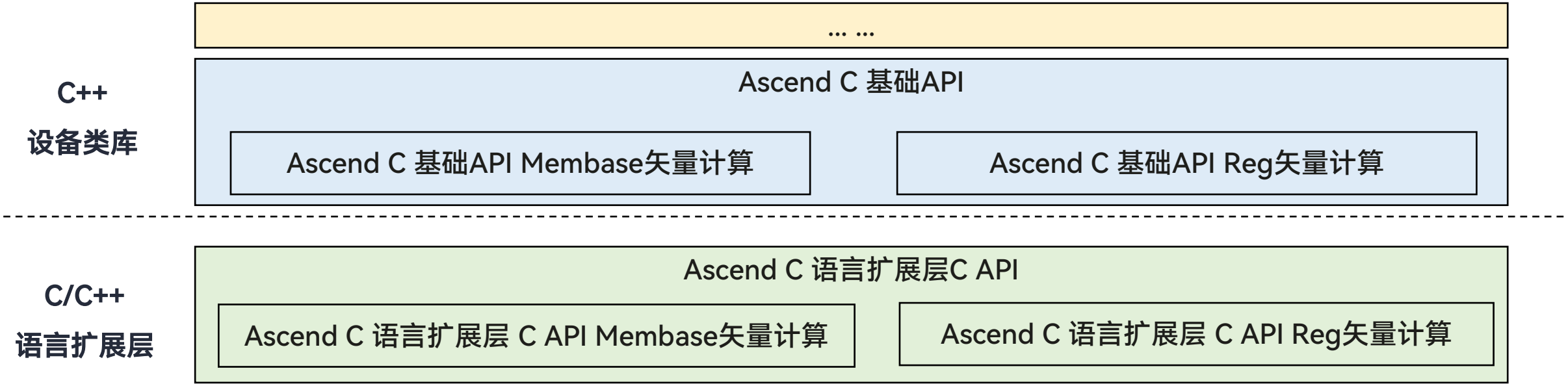
Part 6 当代矢量算子与下一代对比

下一代Reg矢量编程API

Reg矢量编程API在设备类库和语言扩展层均提供了相应的接口，其中：

- Ascend C基础API Reg矢量计算提供C++ 模板化函数
- Ascend C语言扩展层C API Reg矢量计算提供C风格的API接口

在功能上Ascend C基础API 和Ascend C语言扩展层C API相同，可以相互替代



为了保持与当前编程体系兼容，在下一代同时提供Membase矢量计算接口。

CANN

C API – 寄存器类型

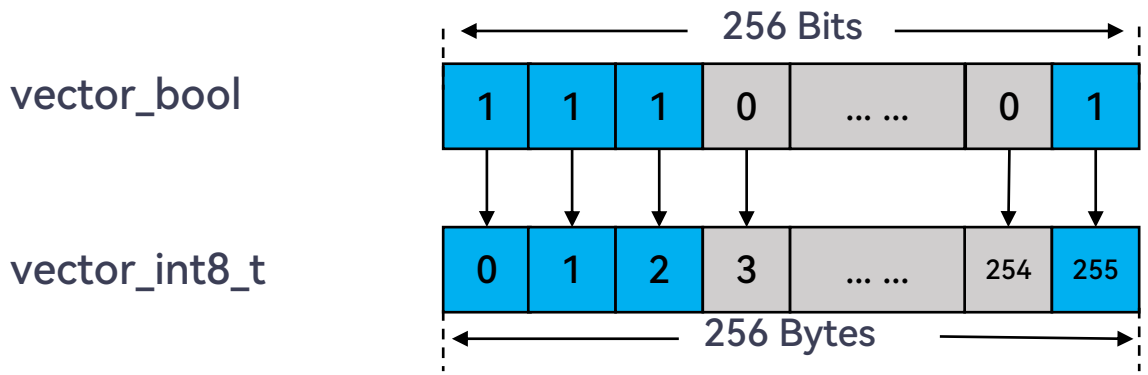
矢量寄存器（寄存器长度 = 1 x VL Bytes）	
单有效元素位宽	矢量数据类型
b8	vector_int8_t、vector_uint8_t、 vector_int4x2_t、vector_fp4x2_e2m1_t、vector_fp4x2_e1m2_t、 vector_hifloat8_t、vector_fp8_e4m3fn_t、vector_fp8_e5m2_t、vector_fp8_e8m0_t
b16	vector_int16_t、vector_uint16_t、vector_half、vector_bfloat16_t
b32	vector_int32_t、vector_uint32_t、vector_float
b64	vector_int64_t、vector_uint64_t

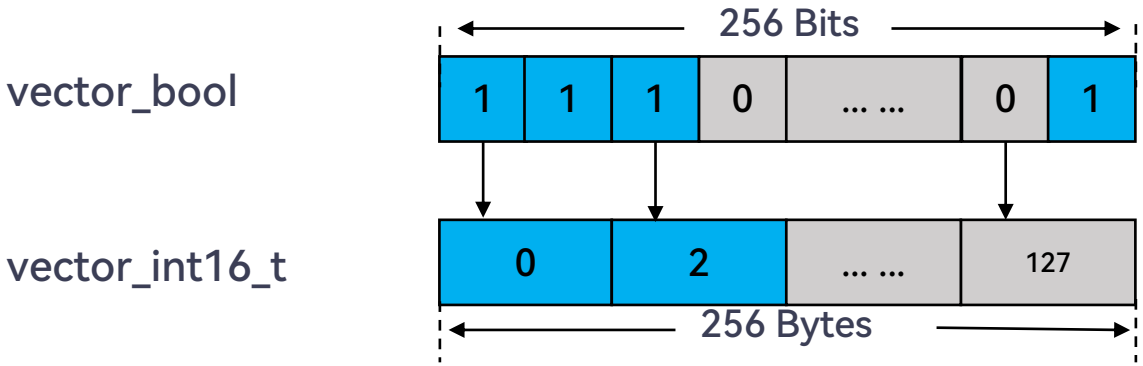
说明： vector_int4x2_t、vector_fp4x2_e2m1_t、vector_fp4x2_e1m2_t这三个矢量数据类型在内存中的排布需要将两个元素打包为一个字节的存储单元。

矢量寄存器在SIMD VF函数内定义时，可以直接定义，具体分配的物理寄存器由编译器自动完成

C API – 寄存器类型

掩码寄存器(寄存器长度 = 1 x VL / 8 Bytes)	
单元素位宽	矢量数据类型
b1	vector_bool

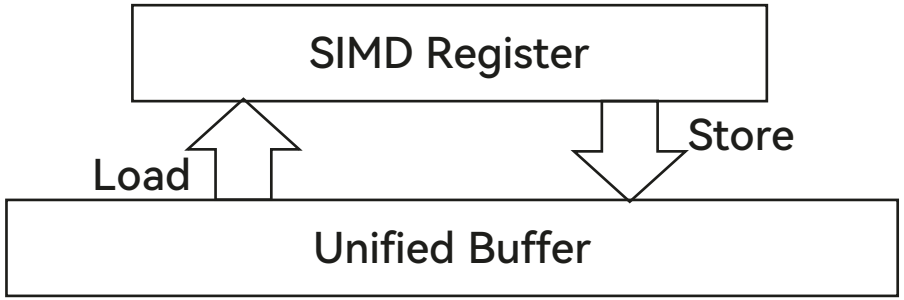




掩码寄存器按照1bit对应矢量寄存器一个最小元素1Byte，长度为 64Bytes 即256bits.

在计算不同位宽数据时，掩码寄存器生效bit如右图所示，为字节严格对应。

C API – Reg搬运



Reg搬运是指UB和Register之间的数据传输。其中从UB到Regster的操作为Load(asc_load），从Regster到UB的操作为Store(asc_store)

非对齐搬运

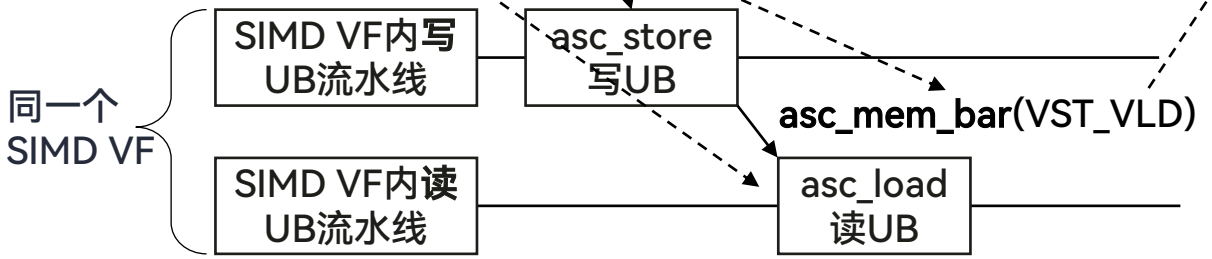
Reg搬运进一步区分对齐搬运和非对齐搬运两种接口。对齐是指UB起始地址是32的整数倍，否则为非对齐。

- 读写对齐地址，使用asc_load_align / asc_store_align接口
- 在读非对齐地址前，应该通过asc_loadunalign_pre，然后再使用asc_loadunalign
- 在写非对齐地址时，应先使用asc_storeunalign，再使用asc_storeunalign_post进行处理
- 无论对齐或者非对齐，均可以使用asc_load、asc_store接口

C API – Reg同步

- **在一个SIMD_VF内对同一片UB地址读写操作**，需要开发者根据数据依赖关系在插入同步接口asc_mem_bar(MEM_TYPE mem_type)

```
asc_store(ub_addr, vreg0, mask);
asc_mem_bar(VST_VLD);
asc_load(vreg2, ub_addr);
```



- **在一个SIMD_VF内对同一个寄存器**进行读写操作时，同步关系由系统自动保证，无需开发者插入同步。

计算结果写Reg寄存器

```
asc_mul(vreg1, vreg0, vreg0, mask);
asc_mul(vreg2, vreg1, vreg0, mask);
```

计算源数据读Reg寄存器

MEM_TYPE值	源流水线	目的流水线
VV_ALL	VEC_ALL	VEC_ALL
VST_VLD	VEC_STORE	VEC_LOAD
VLD_VST	VEC_LOAD	VEC_STORE
VST_VST	VEC_STORE	VEC_STORE
VS_ALL	VEC_ALL	SCALAR_ALL
VST_LD	VEC_STORE	SCALAR_LOAD
VLD_ST	VEC_LOAD	SCALAR_STORE
VST_ST	VEC_STORE	SCALAR_STORE
SV_ALL	SCALAR_ALL	VEC_ALL
ST_VLD	SCALAR_STORE	VEC_LOAD
LD_VST	SCALAR_LOAD	VEC_STORE
ST_VST	SCALAR_STORE	VEC_STORE
SS_ALL	SCALAR_ALL	SCALAR_ALL
ST_LD	SCALAR_STORE	SCALAR_LOAD
LD_ST	SCALAR_LOAD	SCALAR_STORE
ST_ST	SCALAR_STORE	SCALAR_STORE

C API 和基础API Reg矢量计算接口对比

C++ 基础API Reg矢量计算接口提供功能与C API接口功能一致，匹配不同开发者的习惯，部分映射关系如下：

类型	C API Reg矢量计算	C++ 基础API Reg矢量计算接口
寄存器类型	vector_float	AscendC::Reg::RegTensor<float>
	vector_half	AscendC::Reg::RegTensor<half>
	vector_int16_t	AscendC::Reg::RegTensor<int16_t>
Reg搬运	asc_load	AscendC::Reg::Load
	asc_loadalign	AscendC::Reg::LoadAlign
	asc_storeunlign	AscendC::Reg::StoreUnalign
Reg同步	asc_mem_bar	AscendC::Reg::LocalMemBar
Reg矢量计算	asc_add	AscendC::Reg::Add
	asc_mul	AscendC::Reg::Mul
	asc_float2int32	AscendC::Reg::Cast
	asc_half2bfloat16	

本章小结

- Ascend C支持下一代AIV的SIMD Reg架构编程：
  - 新增C 语言扩展Reg矢量计算API
  - 新增基础API Reg矢量计算API

- Ascned C 开放Reg编程增强特性：
  - 开放SIMD Reg矢量数据类型定义和编程
  - 新增支持SIMD VF对UB非对齐访问
  - SIMD VF内部的寄存器依赖同步由硬件保证不需要插入同步
  - SIMD VF读写UB可能存在数据依赖，此时需要开发者插入asc_mem_bar保证数据读写正确
  - SIMD Reg寄存器直接定义，具体分配由编译器自动完成

目录

Part 1 课程背景和目标

Part 2下一代SIMD抽象Regbase硬件架构

Part 3下一代Reg矢量编程模型

Part 4下一代Reg矢量编程API

Part 5下一代Reg矢量编程优化技巧

Part6 当代矢量算子与下一代对比

GELU算子示例

高斯误差线性单元激活函数精确公式如下：

$$GELU(x) = x \cdot \Phi(x)$$

其中$\Phi(x)$ 是标准正态分布的累积分布函数（CDF）。

当将tanh作为approximate，则近似公式如下：

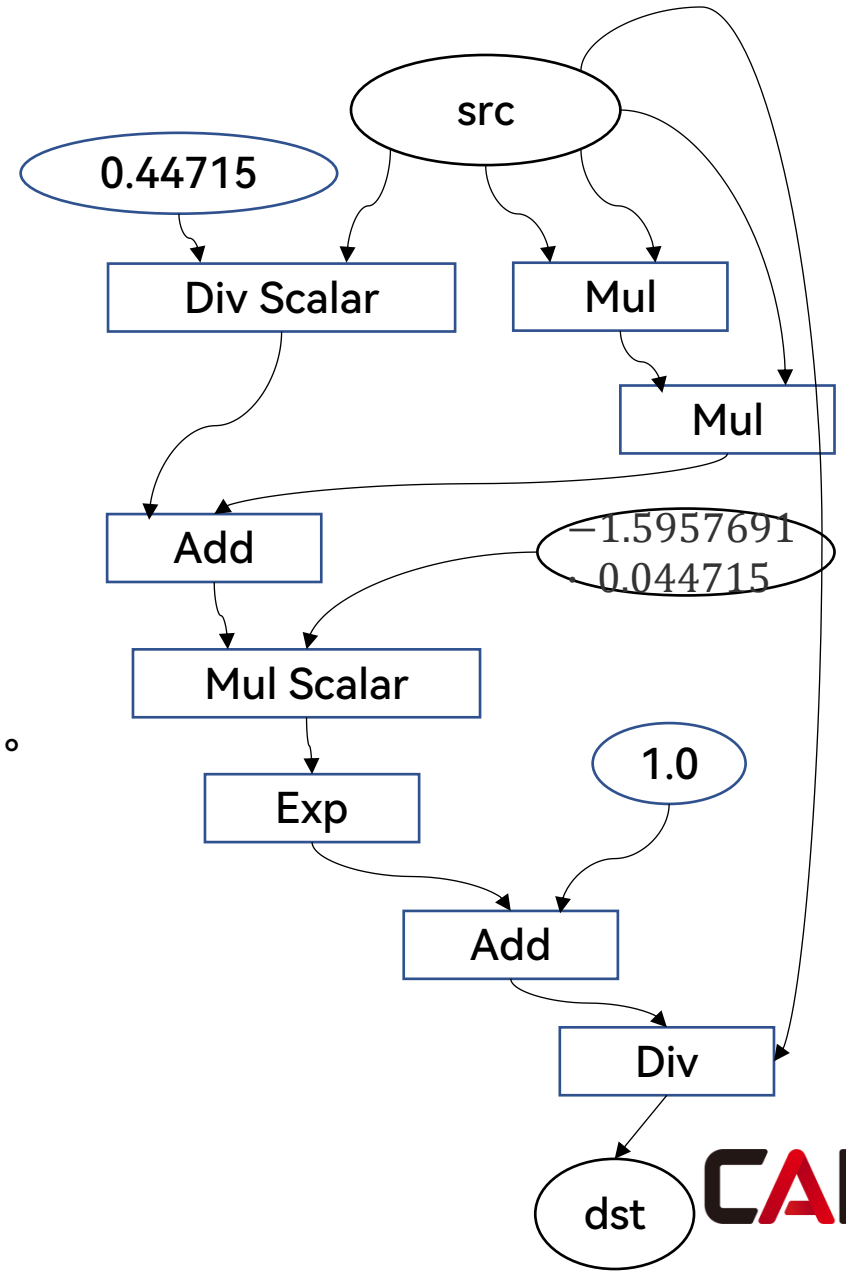
$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{2/\pi} \cdot \left(x + 0.044715 \cdot x^3\right)\right)\right)$$

$$\approx x/(1 + e^{-1.5957691 \cdot 0.044715 \cdot (\frac{x}{0.044715} + x^3))})$$

从计算图上可以看出，需要实现GELU的时候，需要使用多个API完成。

步骤如下：

- **步骤一**：src数据在GlobalMemory上，从GM->UB搬运
- **步骤二**：实现矢量计算SIMD VF
  1. 定义一个SIMD VF，并把数据从UB搬运到Reg
  2. 调用Reg矢量计算API基于Reg完成计算
  3. 将计算结果从Reg写回UB
- **步骤三**：将计算结果，从UB->GM搬运



CANN

Reg矢量编程

以计算Mul、Mul Scalar节点为例，需要开发两个VF，asc_mul_vf 、asc_mul_scalar_vf,通过矢量编程实现如下：

```
__simd_vf__ inline void asc_mul_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src0_local_addr,
    __ubuf__ float* src1_local_addr, uint32_t count) {
    vector_float vreg0, vreg1, vreg2;
    vector_bool mask;
    uint32_t loop_num = ceil(count / 64); // 256B包含64个float
    for (uint32_t i = 0; i < loop_num; i++) {
        mask = asc_updatemask_b32(count);

        asc_load(vreg0, src0_local_addr + i * 64);
        asc_load(vreg1, src1_local_addr + i * 64);
        asc_mul(vreg2, vreg0, vreg1, mask);
        asc_store(dst_local_addr + i * 64, vreg2, mask);
    }
}
```

```
__simd_vf__ inline void asc_mul_scalar_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src0_local_addr,
    __ubuf__ float scalar, uint32_t count) {
    vector_float vreg0, vreg1;
    vector_bool mask;
    uint32_t loop_num = ceil(count / 64); // 256B包含64个float
    for (uint32_t i = 0; i < loop_num; i++) {
        mask = asc_updatemask_b32(count);

        asc_load(vreg0, src0_local_addr + i * 64);
        asc_mul_scalar(vreg1, vreg0, scalar, mask);
        asc_store(dst_local_addr + i * 64, vreg1, mask);
    }
}
```

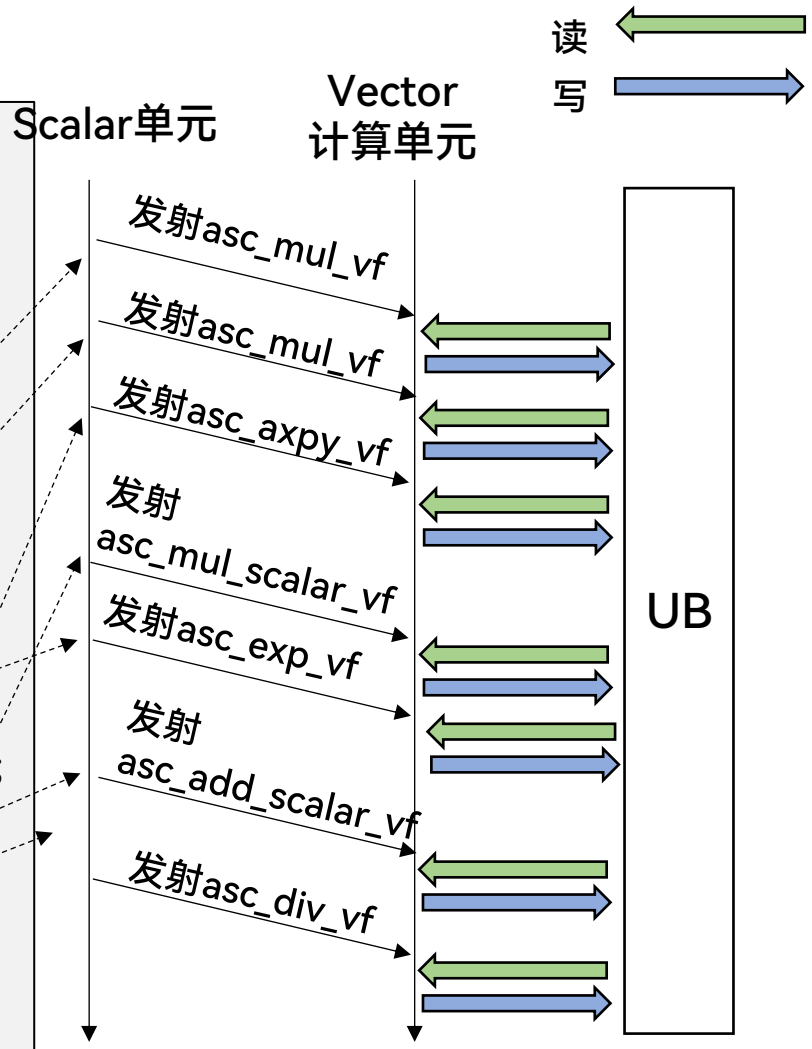
1. 定义一个SIMD VF，并把数据从UB搬运到Reg
2. 调用Reg矢量计算API基于Reg完成计算
3. 将计算结果从Reg写回UB

- VF内Reg矢量每次循环计算一个VL = 256Bytes长度；使用掩码寄存器mask，保证只有有效的数据参与计算。

CANN

Reg矢量编程优化技巧 VF融合

```
...
uint32_t loop_num = floor(count / tile_size);
for (uint32 i = 0; i <= loop_num; i++) {
    ......
    __ubuf__ float* src_local_addr = (__ubuf__ float*)srcLocalTensor.GetPhyAddr();
    __ubuf__ float* tmp1_addr = (__ubuf__ float*)dstLocalTenosr.GetPhyAddr();
    ......
    AscendC::DataCopy(srcLocalTensor, srcGlobalTenosr[ i * tile_size], tile_size);
    ......
    asc_vf_call<asc_mul_vf>(tmp0_addr, src_local_addr, src_local_addr, tile_size);
    asc_vf_call< asc_mul_vf >(tmp1_addr, tmp0_addr, src_local_addr, tile_size);
    asc_vf_call< asc_axpy_vf >(tmp1_addr, src_local_addr, (float)(1 / 0.044715), tile_size);
    asc_vf_call< asc_mul_scalar_vf >(tmp1_addr, (float)(-1.595769121 * 0.044715), tile_size);
    asc_vf_call< asc_exp_vf >(tmp1_addr, tmp1_addr, tile_size);
    asc_vf_call< asc_add _scalar_vf >(tmp1_addr, tmp1_addr, (float)(1.0, tile_size);
    asc_vf_call< asc_div_vf >(tmp1_addr, src_local_addr, tmp1_addr, tile_size);
    ......
    AscendC::DataCopy(dstGlobalTensor[ i * tile_size], dstLocalTenosr, tile_size);
    ......
}
...
```



当对所有的计算节点完成开发后，则Kernel示例

每一个vf均需要Scalar单元发射一次vf，每一个vf均需要读写一次UB

CANN

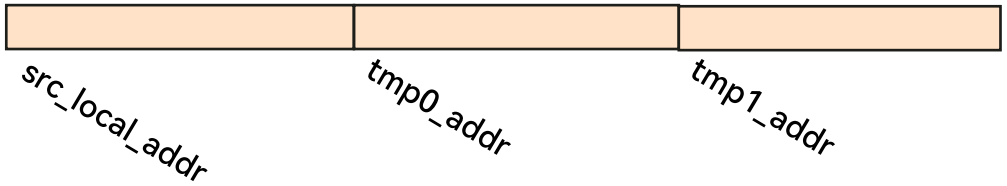
Reg矢量编程优化技巧 VF融合

从上述的代码中可以看到需要UB的空间需要三块：

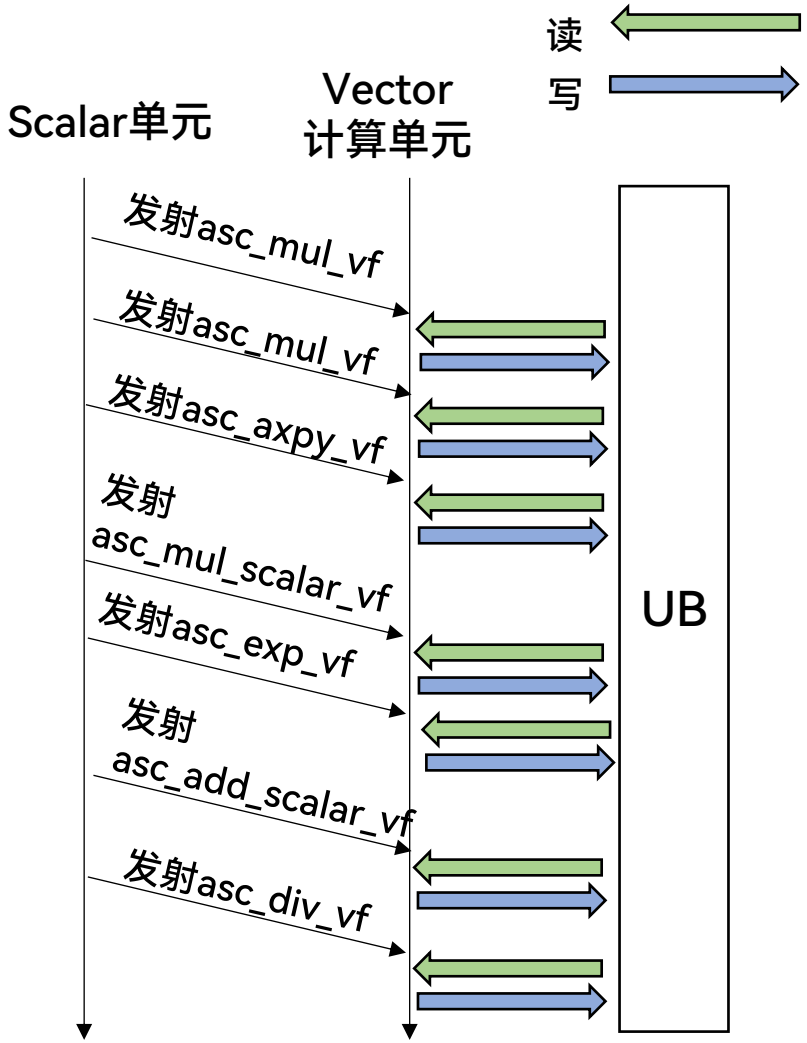
src_local_addr/ tmp0_addr / tmp1_addr

假如UB大小为 256KB时，最多一次循环可以计算tile_size = 256KB / 3 ≈ 85KB；

同时，每个SIMD VF都需要读写UB，会因为UB读写带来计算的bubble。



从示意图中够可以看到，Scalar单元需要发射7次VF，同时需要读UB7次，写UB7次。



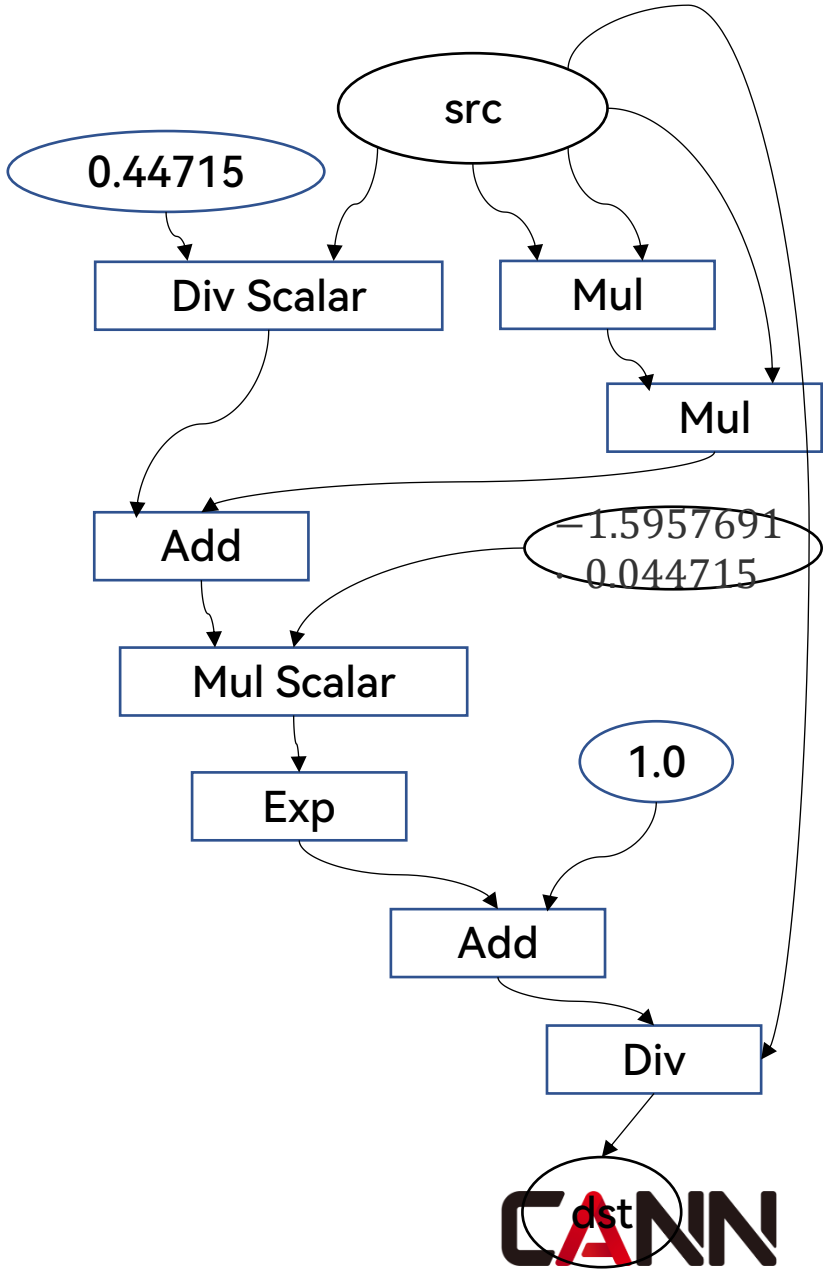
基于Reg矢量编程可以通过多个VF进行融合，进而减少Scalar发射VF开销和读写UB开销。

CANN

Reg矢量编程优化技巧 VF融合

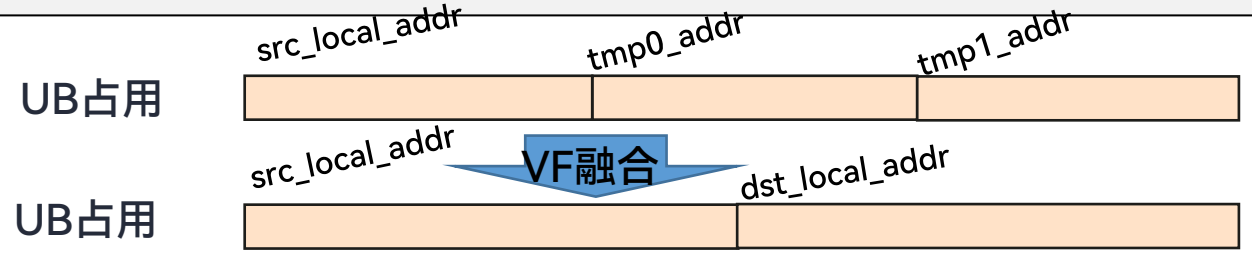
通过前序的计算图，可以将写出如下的一个VF融合后的SIMD VF

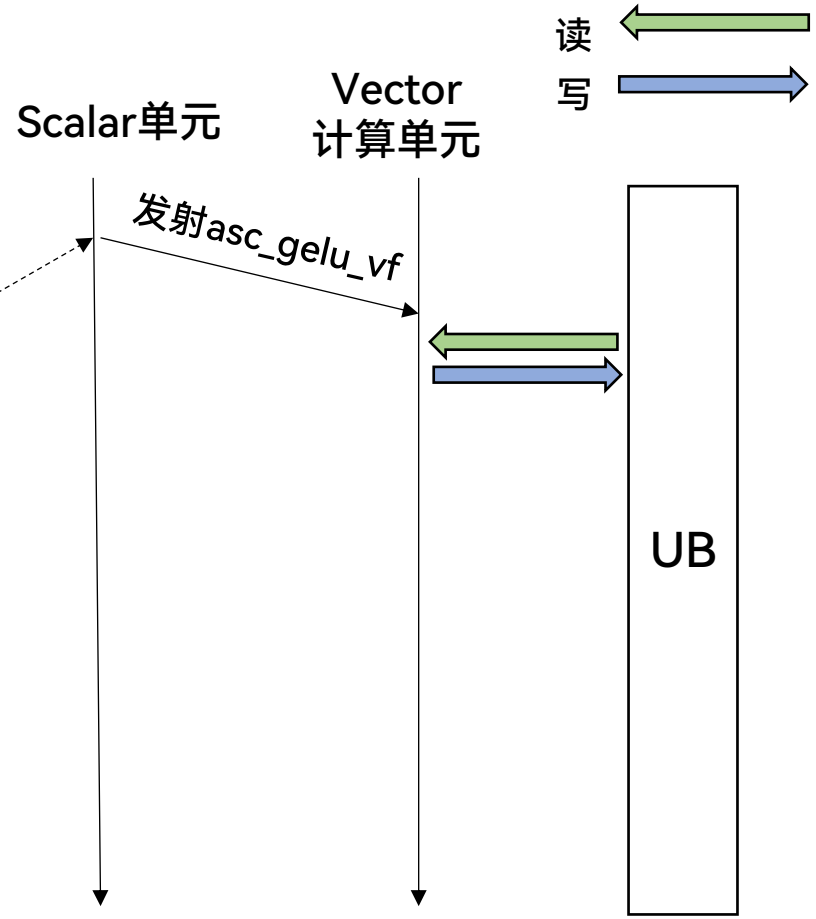
```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    vector_float vreg0, vreg1, vreg2;
    vector_bool mask;
    uint32_t loop_num = ceil(count / 64); // 256B包含64个float
    for (uint32_t i = 0; i < loop_num; i++) {
        mask = asc_updatemask_b32(count);

        asc_load(vreg0, src_local_addr + i * 64);
        asc_mul(vreg1, vreg0, vreg0, mask);
        asc_mul(vreg2, vreg1, vreg0, mask);
        asc_axpy(vreg2, vreg0, (float)(1 / 0.044715), mask);
        asc_mul_scalar(vreg2, vreg2, (float)(-1.595769121 * 0.044715), mask);
        asc_exp(vreg2, vreg2, mask);
        asc_add_scalar(vreg2, vreg2, (float)1.0, mask);
        asc_div(vreg2, vreg0, vreg2, mask);
        asc_store(dst_local_addr + i * 64, vreg2, mask);
    }
    .....
}
```



Reg矢量编程优化技巧 VF融合

```
...
uint32_t loop_num = floor(count, tile_size);
for (uint32 i = 0; i < loop_num; i++) {
    ......
    AscendC::DataCopy(srcLocalTensor, srcGlobalTenosr[ i * tile_size], tile_size);
    ......
    asc_vf_call<asc_gelu_vf>(dst_local_addr, src_local_addr, tile_size);
    ......
    AscendC::DataCopy(dstGlobalTensor[ i * tile_size], dstLocalTenosr, tile_size);
    ......
}
...
```





- 临时数据直接在寄存器交换，消除了中间临时UB的占用，提升有效数据吞吐
- 临时数据直接在寄存器交换，减少每次计算对UB的读写操作开销
- 计算进行VF融合，减少VF发射次数，减少mainscalar开销

CANN

Reg矢量编程优化技巧 对齐搬运

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    for (uint32_t i = 0; i < loop_num; i++) {
        ...
        asc_load(vreg0, src_local_addr + i * 64);
        ...
        asc_store(dst_local_addr + i * 64), vreg2, mask);
    }
}
```

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    for (uint32_t i = 0; i < loop_num; i++) {
        ...
        asc_loadalign(vreg0, src_local_addr + i * 64);
        ...
        asc_storealign(dst_local_addr + i * 64), vreg2, mask);
    }
}
```

- UB对齐地址(可以被32整除)，可以使用高性能对齐搬运接口(asc_loadalign / asc_store_align)

CANN

Reg矢量编程优化技巧 PostUpdate

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    for (uint32_t i = 0; i < loop_num; i++) {
        ...
        asc_loadalign(vreg0, src_local_addr + i * 64);
        ...
        asc_storealign(dst_local_addr + i * 64), vreg2, mask);
    }
}
```

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    for (uint32_t i = 0; i < loop_num; i++) {
        ...
        asc_loadalign_postupdate(vreg0, src_local_addr, 64);
        ...
        asc_storealign_postupdate(dst_local_addr, vreg2, 64, mask);
    }
}
```

操作UB的地址由开发者根据每次读写的长度进行偏移，每次循环偏移64，计算完成后src_local_addr、dst_local_addr不变

UB的地址会被硬件每次循环自动偏移64，计算完成后src_local_addr、 dst_local_addr会更新为最后一次循环操作的地址

- 使用postupdate搬运模式，可以减少scalar计算，减少AuxScalar硬件单元开销

CANN

Reg矢量编程优化技巧 HardwareLoop

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    uint32_t loop_num = ceil(count / 64); // 256B包含64个float
    for (uint32_t i = 0; i < loop_num; i++) {
        ...
    }
}
```

```
__simd_vf__ inline void asc_gelu_vf(
    __ubuf__ float* dst_local_addr,
    __ubuf__ float* src_local_addr, uint32_t count) {
    ...
    uint16_t loop_num = ceil(count / 64); // 256B包含64个float
    for (uint16_t i = 0; i < loop_num; i++) {
        ...
    }
}
```

- SIMD VF内的迭代变量使用位宽小于等于uint16_t数据类型表示时，硬件会使用HardwareLoop功能代替SoftwareLoop，可以极大提升VF执行性能。

硬件使用HardwareLoop其他主要限制：

1、循环起始值必须为0

2、循环步长必须为1

3、循环内无if等跳转指令（可以使用for(1)代替）等

CANN

Reg矢量编程示例 C++基础API Reg矢量编程示例

```
template<class T>
__simd_vf__ void inline GeluVF(__ubuf__ T* srcAddr, __ubuf__ T* dstAddr, uint32_t count) {
    uint32_t dtypeSize = sizeof(T);
    uint32_t vl = VECTOR_LENGTH / dtypeSize;
    uint16_t loopNum = CeilDivision(count, vl);
    uint32_t vlSize = vl;
    AscendC::Reg::RegTensor<T, AscendC::Reg::RegTraitNumOne> vregInput;
    AscendC::Reg::RegTensor<T, AscendC::Reg::RegTraitNumOne> vregInputSqr;
    AscendC::Reg::RegTensor<T, AscendC::Reg::RegTraitNumOne> vregInputCub;
    AscendC::Reg::RegTensor<T, AscendC::Reg::RegTraitNumOne> vregOutput;
    AscendC::Reg::MaskReg mask;
    if constexpr(std::is_same_v<T, float>) {
        for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
            mask = AscendC::Reg::UpdateMask<T, AscendC::Reg::RegTraitNumOne>(count);

            AscendC::Reg::Load(vregInput, (__ubuf__ T*)(srcAddr + loopIdx * vlSize));
            AscendC::Reg::Mul(vregInputSqr, vregInput, vregInput, mask);
            AscendC::Reg::Mul(vregInputCub, vregInputSqr, vregInput, mask);
            AscendC::Reg::Axpy(vregInputCub, vregInput, TANH_APPROX_FACTOR, mask);
            AscendC::Reg::Muls(vregInputCub, vregInputCub, NEG_SQRT_EIGHT_OVER_PI, mask);
            AscendC::Reg::Exp(vregInputCub, vregInputCub, mask);
            AscendC::Reg::Adds(vregInputCub, vregInputCub, (float)1.0, mask);
            AscendC::Reg::Div(vregOutput, vregInput, vregInputCub, mask);

            AscendC::Reg::Store((__ubuf__ T*)(dstAddr + loopIdx * vlSize), vregOutput, mask);
        }
    }
}
```

前序介绍了使用C API的reg矢量计算，同期发布的C++基础API Reg计算同样可以使用。

CANN

https://gitcode.com/cann/ops-nn/blob/master/activation/gelu/op_kernel/arch35/gelu_dag.h

本章小结

- 下一代Reg编程方式：
  - 新增VF融合、对齐搬运、PostUpdate、使能HardwareLoop等优化手段

CANN

目录

Part 1课程背景和目标

Part 2下一代SIMD抽象Regbase硬件架构

Part 3下一代Reg矢量编程模型

Part 4下一代Reg矢量编程API

Part 5下一代Reg矢量编程优化技巧

Part 7 当代矢量算子与下一代对比

当代矢量编程与下一代编程对比示例

当代和下一代均支持基础 API Membase矢量计算API编程

```
...
uint32_t loop_num = floor(count / tile_size);
for (uint32 i = 0; i <= loop_num; i++) {
    AscendC::DataCopy(
        srcLocalTensor, srcGlobalTenosr[i * tile_size], tile_size);
    ......
    AscendC::Mul(
        tmp0Local, srcLocalTensor, srcLocalTensor, tile_size);
    AscendC::Mul(tmp1Local, tmp0Local, srcLocalTensor, tile_size);
    AscendC::Axpy(
        tmp1Local, src_local_addr, (float)(1 / 0.044715), tile_size);
    AscendC::Muls(
        tmp1Local, (float)(-1.595769121 * 0.044715), tile_size);
    AscendC::Exp(tmp1Local, tmp1Local, tile_size);
    AscendC::Adds(tmp1Local, tmp1Local, (float)(1.0, tile_size);
    AscendC::Div(tmp1Local, srcLocalTensor, tmp1Local, tile_size);
    ......
    AscendC::DataCopy(
        dstGlobalTensor[i * tile_size], dstLocalTensor, tile_size);
}
...
```

仅下一代支持 基于C++基础 API Reg矢量计算API编程

```
...
for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
    ......
    AscendC::Reg::Load(
        vregInput, (__ubuf__ T*)(srcAddr + loopIdx * vlSize));
    AscendC::Reg::Mul(vregInputSqr, vregInput, vregInput, mask);
    AscendC::Reg::Mul(vregInputCub, vregInputSqr, vregInput, mask);
    AscendC::Reg::Axpy(
        vregInputCub, vregInput, TANH_APPROX_FACTOR, mask);
    AscendC::Reg::Muls(
        vregInputCub, vregInputCub, NEG_SQRT_EIGHT_OVER_PI, mask);
    AscendC::Reg::Exp(vregInputCub, vregInputCub, mask);
    AscendC::Reg::Adds(vregInputCub, vregInputCub, (float)1.0, mask);
    AscendC::Reg::Div(vregOutput, vregInput, vregInputCub, mask);

    AscendC::Reg::Store(
        (__ubuf__ T*)(dstAddr + loopIdx * vlSize), vregOutput, mask);
}
...
```

本章小结

- 当代AIV 矢量编程和下一代区别：
  - 下一代提供C++ 基础API Membase接口
  - Reg矢量编程仅在下一代支持

CANN

附录

- Ascend C SIG: https://gitcode.com/cann/community/blob/master/CANN/sigs/Ascend C/README.md
- Ascend C 核心仓：https://gitcode.com/cann/asc-devkit
- CANN Ascend C 算子仓/算子模板库仓：https://gitcode.com/cann/
- Ascend C 编程指南：https://www.hiascend.com/cann/ascend-c

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN